A Behavior cloning

We collected trajectories from Amazon Mechanical Turk. We removed trajectories that fell short of the intended human-human trajectory length of $T \approx 1200$ and very suboptimal ones (with reward roughly below to what could be achieved by one human on their own – i.e. less than 220 for Cramped Room, 280 for Asymmetric Advantages Ring, 150 for Coordination Ring, 160 for Forced Coordination, and 180 for Counter Circuit). After removal, we had 16 joint human-human trajectories for Cramped Room environment, 17 for Asymmetric Advantages, 16 for Coordination Ring, 12 for Forced Coordination, and 15 for Counter Circuit.

We divide the joint trajectories into two groups randomly, and split each joint trajectory $((s_0, (a_0^1, a_0^2), r_0), \ldots, (s_T, (a_T^1, a_T^2), r_T))$ into two single agent trajectories: $((s_0, a_0^1, r_0), \ldots, (s_T, a_T^1, r_T))$ and $((s_0, a_0^2, r_0), \ldots, (s_T, a_T^2, r_T))$. At the end of this process we have twice as many single-agent trajectories than the joint human-human trajectories we started with, for a total of approximately 36k environment timesteps for each layout.

We then use the two sets of single-agent human trajectories to train two human models, BC and H_{Proxy}, for each of the five layouts. We evaluate these trained behavior cloning models by pairing them with themselves averaging out reward across 100 rollouts with horizon $T = 400$.

In each subgroup, we used 85% of the data for training the behavior cloning model, and 15% for validation. To learn the policy, we used a feed-forward fully-connected neural network with 2 layers of hidden size 64. We report the hyperparameters used in Table 1. We run each experiment with 5 different seeds, leading to 5 BC and 5 H_{Proxy} models for each layout. We then manually choose one BC and one H_{Proxy} model based on the heuristic that the H_{Proxy} model should achieve slightly higher reward than the BC model – to make our usage of H_{Proxy} as a human proxy more realistic (as we would expect BC to underperform compared to the expert demonstrator).

Behavior cloning, unlike all other methods in this paper, is trained on a manually designed 64-dimensional featurization the state to incentivize learning policies that generalize well in spite of the limited amount of human-human data. Such featurization contains the relative positions to each player of: the other player, the closest onion, dish, soup, onion dispenser, dish dispenser, serving location, and pot (one for each pot state: empty, 1 onion, 2 onions, cooking, and ready). It also contains boolean values encoding the agent orientation and indicating whether the agent is adjacent to empty counters. We also include the agent's own absolute position in the layout.

To correct for a tendency of the learnt models to sometimes get stuck when performing low level action tasks, we added a hardcoded the behavior cloning model to take a random action if stuck in the same position for 3 or more consecutive timesteps. As far as we could tell, this does not significantly affect the behavior of the human models except in the intended way.

Behavior cloning hyperparameters					
Parameter	Cramped Rm.	Asym. Adv.	Coord. Ring	Forced Coord.	Counter Circ.
Learning Rate	1e-3	1e-3	1e-3	1e-3	1e-3
# Epochs	100	120	120	90	110
Adam epsilon	1e-8	1e-8	1e-8	1e-8	1e-8

Table 1: Hyperparameters for behavior cloning across the 5 layouts. *Adam epsilon* is the choice of the ϵ for the Adam optimizer used in these experiments.

B Self-play PPO

Unlike behavior cloning, PPO and other DRL methods were trained with a lossless state encoding consisting of 20 masks, each a matrix of size corresponding to the environment terrain grid size. Each mask contains information about a specific aspect of the state: the player's own position, the player's own orientation, location of dispensers of various types, location of objects on counters, etc.

In order to speed up training, we shaped the reward function to give agents some reward when placing an onion into the pot, when picking up a dish while a soup is cooking, and when picking up a soup

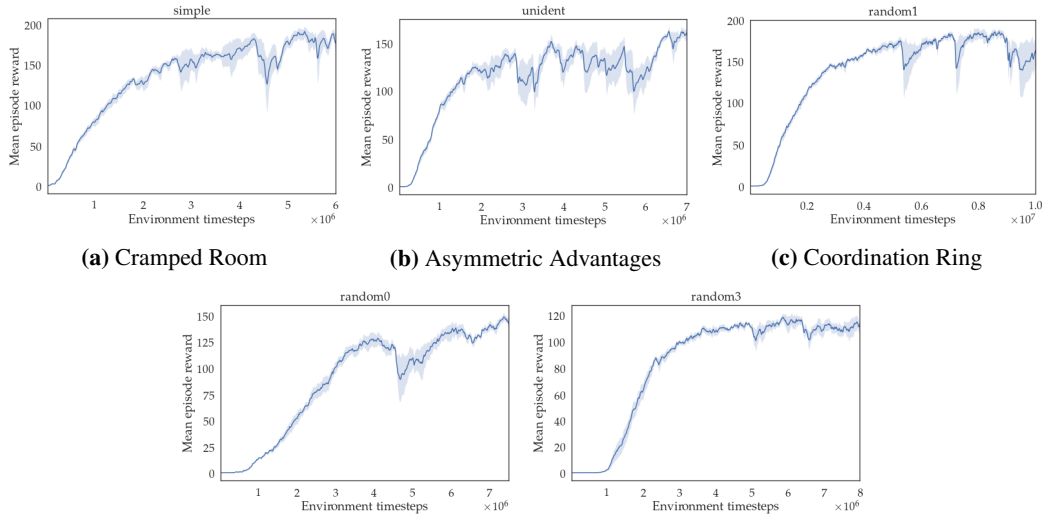
with a dish. The amount of reward shaping is reduced to 0 over the course of training with a linear schedule.

We parameterize the policy with a convolutional neural network with 3 convolutional layers (of sizes 5×5, 3×3, and 3×3 respectively), each of which has 25 filters, followed by 3 fully-connected layers with hidden size 32. Hyperparameters used and training curves are reported respectively in Table 2 and Figures 8.

We use 5 seeds for our experiments, with respect to which we report all our standard errors.

PPO$_{SP}$ hyperparameters					
Parameter	Cramped Rm.	Asym. Adv.	Coord. Ring	Forced Coord.	Counter Circ.
Learning Rate	1e-3	1e-3	6e-4	8e-4	8e-4
VF coefficient	0.5	0.5	0.5	0.5	0.5
Rew. shaping horizon	2.5e6	2.5e6	3.5e6	2.5e6	2.5e6
# Minibatches	6	6	6	6	6
Minibatch size	2000	2000	2000	2000	2000

Table 2: Hyperparameters for PPO trained purely in self-play, across the 5 layouts. For simulation, we used 30 parallel environments. Similarly to the embedded human model case, parameters common to all layouts are: entropy coefficient $(= 0.1)$, gamma $(= 0.99)$, lambda $(= 0.98)$, clipping $(= 0.05)$, maximum gradient norm $(= 0.1)$, and gradient steps per minibatch per PPO step $(= 8)$. For a description of the parameters, see Table 3.



(a) Cramped Room **(b)** Asymmetric Advantages **(c)** Coordination Ring

(d) Forced Coordination **(e)** Counter Circuit

Figure 8: PPO$_{SP}$ self-play average episode rewards on each layout during training.

C PPO with embedded-agent environment

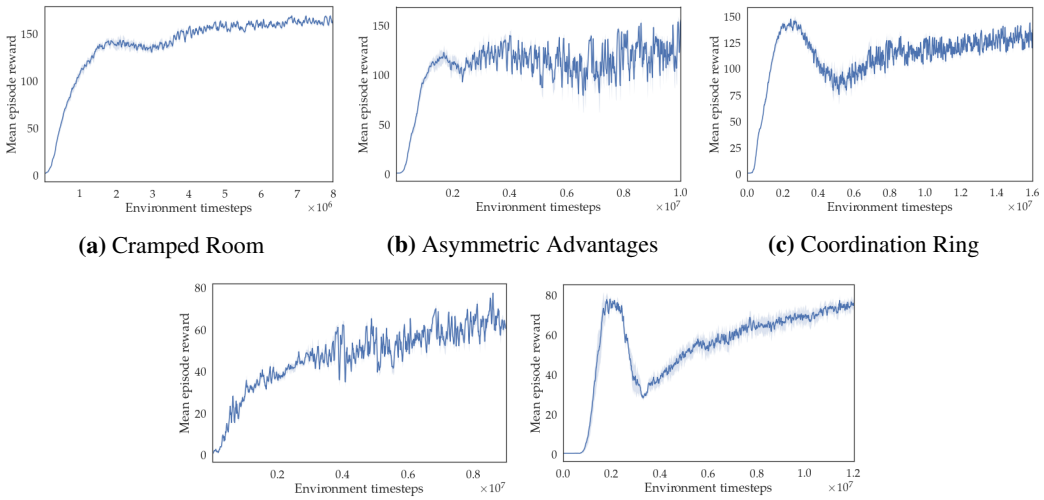
To train PPO with an embedded human model we use the same network structure as in the PPO$_{SP}$ case in Appendix B and similar hyperparameters, reported in Table 3. As in the PPO$_{SP}$ case, we use reward shaping and anneal it linearly throughout training.

Empirically, we found that – for most layouts – agents trained directly with competent human models to settle in local optima, never developing good game-play skills and letting the human models collect reward alone. Therefore, on all layouts except Forced Coordination, we initially train in pure self-play, and then anneal the amount of self-play linearly to zero, finally continuing training purely with the human model. We found this to improve the trained agents' performance. In Forced Coordination, both players need to learn game-playing skills in order to achieve any reward, so this problem doesn't occur.

Training curves for the training (BC) and test (H_{Proxy}) models are reported respectively in Figures 9 and 10.

PPO_{BC} and $PPO_{H_{Proxy}}$ hyperparameters					
Parameter	Cramped Rm.	Asym. Adv.	Coord. Ring	Forced Coord.	Counter Circ.
Learning Rate	1e-3	1e-3	1e-3	1.5e-3	1.5e-3
LR annealing factor	3	3	1.5	2	3
VF coefficient	0.5	0.5	0.5	0.1	0.1
Rew. shaping horizon	1e6	6e6	5e6	4e6	4e6
Self-play annealing	[5e5, 3e6]	[1e6, 7e6]	[2e6, 6e6]	N/A	[1e6, 4e6]
# Minibatches	10	12	15	15	15
Minibatch size	1200	1000	800	800	800

Table 3: Hyperparameters for PPO trained on an embedded human model environment, across the 5 layouts. *LR annealing factor* corresponds to what factor the learning rate was annealed by linearly over the course of the training (i.e. ending at LR_0/LR_{factor}). *VF coefficient* is the weight to assign to the value function portion of the loss. *Reward shaping horizon* corresponds to the environment timestep in which reward shaping reaches zero, after being annealed lineraly. Of the two numbers reported for *self-play annealing*, the former refers to the environment timestep we begin to anneal from pure self-play to embedded human model training, and the latter to the timestep in which we reach pure human model embedding training. *N/A* indicates that no self-play was used during training. *# Minibatches* refers to the number of minibatches used at each PPO step, each of size *minibatch size*. For simulation, we used 30 parallel environments. Further parameters common for all layouts are: entropy coefficient ($= 0.1$), gamma ($= 0.99$), lambda ($= 0.98$), clipping ($= 0.05$), maximum gradient norm ($= 0.1$), and gradient steps per minibatch per PPO step ($= 8$). For further information, see the OpenAI baselines PPO documentation [8].



(a) Cramped Room **(b)** Asymmetric Advantages **(c)** Coordination Ring

(d) Forced Coordination **(e)** Counter Circuit

Figure 9: PPO_{BC} average episode rewards on each layout during training over 400 horizon timesteps, when pairing the agent with itself or with BC in proportion to the current self-play annealing.

D Population Based Training

We trained population based training using a population of 3 agents, each of which is parameterized by a neural network trained with PPO, with the same structure as in C.

During each PBT iteration, all possible pairings of the 3 agents are trained using PPO, with each agent training on a embedded single-agent MDP with the other PPO agent fixed. PBT selection was conducted by replacing the worst performing agent with a mutated version of the hyperparameters. The parameters that could be mutated are lambda (initialized $= 0.98$), clipping (initialized $= 0.05$), learning rate (initialized $= 5e-3$), gradient steps per minibatch per PPO update (initialized $= 8$),